

Programming for Analytics

Week 1: Introduction to Programming · Text, Math, Variables & Types

Day 1 at The Daily Grind: the shop gets a name, a menu, and its very first receipt.

Romain Claret · University College Dublin, 2026

Where you are

Week 1 of 10

You can already *(from the overview)*: how this course works (café, project in W6) · Python 3 + terminal + VS Code (installing this week)

Today you add: what a program *is* · `print()` · strings · numbers & math · booleans · variables · types & conversions · the float wall

Still ahead: talking & remembering (W2) · decisions & loops (W3) · data structures (W4) · memory & unbreakable code (W5) · numpy + pandas + **your project** (W6, a double week) · classes (W7) · functional + GUIs (W8) · complexity (W9) · testing & exam prep (W10)

§0 · Outline: what's in this file

§	Section	Café / concept	Page
1	What is a program, and why you	CONCEPT	5
2	How computers speak	binary → machine code → Python	8
3	Your first program	CAFÉ: our first line + running it	12
4	Under the hood	objects, `type()`, character codes	18
	Break		20
5	Working with text	escapes, quotes, `+` `*` `len()`	21
6	Math	arithmetic, precedence, division traps	25
7	Floats, and the wall	the café's first bug	29
8	Booleans	`True`, `False`, and their arithmetic	33
9	Variables	naming, rebinding, memory	35
10	Types & the receipt	conversions, CAFÉ build	39
11	Exam & exercises · checklist		45

This week's café feature: the shop's first receipt, printed by our first program. **New concepts:** programs & languages; `print`; comments; `int`/`float`/`str`/`bool`; arithmetic; variables; `type()` and conversions.

Where we're going

This semester we build **one program together:** "The Daily Grind", a café register. One feature per week, live, using only what we've learned so far.

Today the café gets:

- a **name** and an **open sign**,
- three **prices**,
- and its **first printed receipt**.

That's all week-1 tools can build. Next week it starts talking to customers. By week 10 it will have analytics, a GUI, and tests, and you'll have watched every single line arrive.

What is a computer program?

A **sequence of operations** for a computer to execute. Computers cannot function without one.

- **Hardware:** the physical parts: processor, memory, disks, screen.
- **Software:** the instructions that make the hardware do things: the BIOS, the operating system, utilities, games… and everything you'll write here.

Programs run far beyond computers: phones, cameras (firmware), watches, speakers, light bulbs, washing machines, **even coffee makers**. The *Internet of Things*.

Which is convenient: this semester, **we're the ones programming the café.**

§1 · CONCEPT “But I’m a business student!”

Programming belongs to more people than computer scientists. In analytics it shows up as:

- **data analysis:** beyond what a spreadsheet can express;
- **business software:** stock control, accounting;
- **automation:** of the data processing you’d otherwise do by hand;
- **intelligent data manipulation**, and if you get good, advanced modeling, **trading rules** (yes, that’s the group project waiting for you in week 6).

It’s also one of the most employable skills in Ireland.

> *“On n’est jamais si bien servi que par soi-même” (want something done right, do it yourself). That’s this module in one sentence.*

§1 · Hardware, in one slide

Three jobs, whatever the machine:

Job	Examples
Input: getting data in	keyboard, mouse, microphone, webcam, scanner, drives, network
Output: getting results out	screen, speakers, printer, disks, network
System: running your program	power supply, motherboard, **CPU**, **RAM**, the OS, applications

Your program is a guest of the **CPU** and **RAM**, coordinated by the operating system. That's all the hardware theory we need. We're here for the software.

How computers speak: binary

- Chips carry current: it **passes (1)** or it **doesn't (0)**. That's the whole alphabet.
- **Boolean logic** (George Boole): numbers in base 2, operations on true/false values: a perfect match for that hardware.

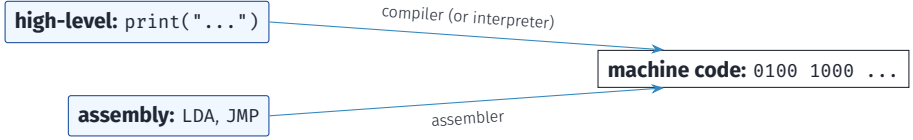
  There are 10 types of people in this world: those who understand binary, and those who don't.

Everything that follows (text, prices, receipts) is, underneath, 0s and 1s.

§2 · CONCEPT From machine code to Python

How do humans write those 0s and 1s?

- **Machine language:** binary instructions, specific to the processor: `C016 B6 80 05` … unreadable, unwriteable.
- **Assembly:** barely human-readable mnemonics (`LDA`, `JMP`), translated by an *assembler*. Still one dialect per processor family.
- **High-level languages:** human-readable, translated by a *compiler* (or interpreter): 1950s FORTRAN · 1960s COBOL, Basic · 1970s C, SQL, Matlab · 1980s C++, Perl · 1990s **Python**, Java, PHP, JavaScript · …



Each step trades raw control for **readability**, and readability is where we live.

Which language for which job?

- **Assembly / C:** maximum speed, minimum comfort (operating systems, embedded).
- **C++ / Java / C#:** large structured applications.
- **Objective-C / Swift:** iPhone apps. **HTML / PHP / JavaScript:** the web.
- **FORTRAN / Matlab / R:** scientific computing. **LaTeX:** beautiful documents.
- **Python:** general-purpose, compact syntax (itself written in C), and the **lingua franca of data work**.

Python has topped the popularity indexes (PYPL) for years, which matters to you practically: more packages, more answers, more jobs.

§2 · CONCEPT Why Python, specifically?

Created by **Guido van Rossum**; high-level, easy to start, hard to outgrow.

Its core philosophy, *the Zen of Python*:

> *Beautiful is better than ugly. Simple is better than complex. Complex is better than complicated.* ***Readability counts.***

We'll say code is *pythonic* (or not) all semester; it means "written the way the language wants to be written."

Who uses it? Google, NASA, the games industry, Industrial Light & Magic, IBM, banks and funds. And: **less time fighting syntax = more time for the actual analytics.** That's why it's ours.

§3 · CAFÉ Your first program

The traditional first program in any language:

```
print("Hello, World!")
```

Ours is:

```
print("The Daily Grind is open for business.")
```

One line. It works. It's tradition, it's a sanity check that your setup runs, and it's the first line of a program you'll still be extending in November.

The same program, elsewhere

C:

```c
#include <stdio.h>
int main() { printf("Hello, World!"); return 0; }
```

Java:

```java
public class HelloWorld {
    public static void main(String[] args) { System.out.println("Hello, World!"); }
}
```

Assembly: ∼15 lines of `mov`/`int`. **Machine code:** a wall of hex bytes.

Same behavior, four languages. Appreciate what one Python line is doing for you.

Anatomy of that one line

```
print("The Daily Grind is open for business.")
```

- `print()` is a **function:** a named operation the language provides.
- Python is **case sensitive**: `print` ≠ `Print` ≠ `PRINT`.
- `"The Daily Grind..."` is a **string literal:** text between `"..."` or `'...'` (the quotes themselves are not printed).

```
print('Open for business.')   # OK: single quotes work too
print('Open for business.")   # BAD: mismatched quotes, SyntaxError
```

§3 · CONCEPT Three ways to run Python

1. **The interpreter:** type `python3` in a terminal (on Windows it's `python`); each line runs when you press Return. Quit with `quit()` or CTRL-D. *(Great for trying things; we'll use >>> for this on slides. The terminal itself: macOS Terminal.app, Windows PowerShell, or easiest, VS Code's menu Terminal → New Terminal, which opens one next to your file.)*
2. **From a file:** write lines into `cafe.py`, run `python3 cafe.py` (Windows: `python cafe.py`); everything executes top to bottom. *(This is how our café runs.)*
3. **From an IDE** (VS Code, PyCharm, Spyder, …): editor + console in one window.

In this class I work with **a terminal + VS Code:** that's the setup the tutors and I support. Notation on my slides: `$` = terminal · `>>>` = Python shell · plain = program file.

§3 · TRY IT Your turn (2 min)

On your machine (any of the three ways):

1. Print the café's opening line.
2. Break it on purpose: try `Print(...)` with a capital P. Read the error.
3. Fix it, and add a **second** print of your own: your café's slogan.

If Python isn't installed yet: that's this week's homework (python.org, Python 3), and the week-2 practical starts from exactly here. *(Stopgap for today only: the browser options from the overview: py4e's exercise pages or Colab.)*

Comments

```python
# Formal and polite
print("The Daily Grind is open for business.")
print("What can I get you?")   # a bit friendlier
```

- Everything after # is a **comment**: not executed, not printed, for humans only.
- Comment *why*, not *what*, and remember the Zen: **readability counts.**

For the experienced in the room: yes, this is where good habits start. A café you can't read in November is a café you can't extend in November.

Everything is an object

```
>>> print(type("The Daily Grind"))
<class 'str'>
```

- In Python, **everything is an object:** text, numbers, even results of calculations.
- `type()` tells you what kind of object you're holding.
- `str` is a **string**: a sequence of characters.

This one function will answer half of your "why doesn't this work?" questions all semester. When in doubt: `type()` it. Its partner is `help()`: `help(str)` prints what a string can *do* (all its methods) and how, straight from the shell. Two ways to ask Python about itself, no web search needed.

What a string really is

Remember: everything in a computer is a **number** (and every number is binary).

```
"Hello, World!"                      what you see
  H   e   l   l   o   ,     W ...     the characters, one per code
 72 101 108 108 111 44 32 87 ...     the character codes (decimal)
 48 65 6c 6c 6f 2c 20 57 ...         the same codes (hexadecimal)
 0100 1000 0110 0101 ...             the same codes (binary)
```

Every character has a **numeric code:** look up “ASCII table”. Examinable, and a recurring exam question: *“what is a string, underneath?”*: a sequence of character codes. Nothing more.

Break

10–15 minutes

*When we come back: the café learns **text tricks**, does its **math**, meets its **first bug**, and prints its **first receipt**.*

Think about it: the receipt prints at the end of today. What pieces of information must a receipt hold?

☎ *Answer on PollEv when we come back.*

Special characters

What if the text itself contains a quote?

```
>>> print("How's it goin', \"World\"?")
How's it goin', "World"?
```

Escape characters: a backslash gives the next character a special meaning:

- `\"` and `\'`: a literal quote;
- `\n`: a **new line**; `\t`: a **tab**.

```
>>> print('Ready?\n\tYes.\n\tNo.')
Ready?
        Yes.
        No.
```

Triple quotes & long lines

```
>>> print('''I'm ok.
... I think.''')
@@I'm ok.
I think.
```

- Between **triple quotes** (`'''` or `"""`) you need no escaping of quotes or line breaks (escape sequences still work), and line breaks are literal. (`...` is the shell's continuation prompt.)
- Escaped, double-quoted, or triple-quoted: **same internal string, same output.**
- A trailing \ continues a long command on the next line (use sparingly).

§5 · CAFÉ String arithmetic

Strings have operators too, and the café uses all three today:

```
>>> print("The Daily " + "Grind")     # + joins strings
The Daily Grind

>>> print("=" * 34)                   # * repeats a string
==================================

>>> print(len("The Daily Grind"))     # len() measures one
15
```

That `"=" * 34` line? That's our **receipt divider**. One character, multiplied, instead of typing 34 equals signs like an animal.

§5 · TRY IT Predict, then run (3 min)

What does each line print? Commit to an answer *before* you try it:

```
print("To break a line, use \\n.")
print("First line."); print("Second line.")
print("First line.\n" + "Second line.")
print("Decaf? Yes.\b\b\b\bNO!!")
print("G" + "o" * 10 + "d morning, Dublin!")
print("Single", end=" "); print("line.")
```

New pieces: `\\` prints a real backslash · `;` chains statements (rarely wise) · `\b` backspaces · `end=" "` changes what `print` puts at the end (default: a newline).

Arithmetic

```
>>> print(2 + 5, 2 - 5, 2 * 5, 2 / 5, 2 % 5)
7 -3 10 0.4 2
```

- `print` takes several values separated by commas.
- `%` is the **remainder** of a division, and the instant even/odd test: `4 % 2` → `0` (even), `5 % 2` → `1` (odd).
- Spaces around operators are ignored, but **use them**; it's the pythonic way.

Powers: `2 ** 5` → 32. No space inside `**`: `2 * * 5` is a `SyntaxError`. Other languages and Excel write powers with ^; in Python ^ is a *different* operator, so `2 ^ 5` gives 7, not 32. Always `**` for powers. And `//` is **floor division**: `3 // 2` → `1` (the division, rounded down).

Precedence: run these by hand

Work each one out on paper **before** Python does:

Expression	Your answer	Python says
`2 + 3 * 4`		`14` (`*` before `+`)
`(2 + 3) * 4`		`20` (brackets first)
`2 - - 1`		`3` (minus a negative)
`2 - - - 1`		`1` (three minuses)
`2 * 2 ** 2 * 2`		`16` (`**` binds tightest)
`3 / 2` vs `3 // 2`		`1.5` vs `1`

When in doubt: **brackets**. They cost nothing and remove every ambiguity.

Division by zero

```
>>> print(2 / 0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

- Division by zero is undefined, so Python **stops your program** with an error.
- This is one of the biggest sources of real-world bugs: an *average over zero orders*, a *rate per zero days*…

The café meets this for real in Week 5, splitting a bill between zero people. Remember the name: `ZeroDivisionError`.

The café does its math

```
espresso_price = 2.50
muffin_price = 2.40

# a customer buys 2 espressos and 1 muffin
total = espresso_price * 2 + muffin_price * 1
print(total)                    # 7.4
```

Prices, quantities, a total: arithmetic is the register's day job. *(Those names on the left? Variables, properly introduced in §9. The math came first because the math is why the register exists.)*

Floats

- `2.0 + 5.0` → `7.0`: float arithmetic works like integer arithmetic.
- **Mixing** int and float always produces a float: `2 - 5.0` → `-3.0`.
- Check with `type()`: `type(2 + 5)` → `int`, `type(2.0 + 5.0)` → `float`.

But floats hide something. Remember: computers are **binary**. A decimal number must be stored as 0s and 1s (IEEE-754: a sign, a fraction, an exponent). Like recording music digitally: almost perfect, **not exact**.

```
>>> print(10 / 3)
3.3333333333333335
```

Seventeen-ish significant figures, then the representation runs out.

The wall: our first bug

A customer orders **three lattes** at 3.20:

```
>>> print(3.20 * 3)
9.600000000000001
```

That is not a typo. The canonical demonstration:

```
>>> print(0.1 + 0.1)
0.2
>>> print(0.1 + 0.1 + 0.1)
0.30000000000000004
```

1/2 and 7/8 store exactly in binary; **1/10 cannot:** tiny errors, and they accumulate. Meanwhile `10/3 + 10/3 + 10/3` → `10.0` (errors can also cancel!). The lesson: **never depend on overly precise float values.**

So is our register broken?

No, but our **display** is naive. Three facts to hold on to:

1. The stored value is off by ∼0.000000000000001: **nobody's coffee money is harmed.**
2. Every calculator and register you've ever used has the same issue; they just **round the display**.
3. We don't have the display tools yet. **Next week we get proper formatting**, and the receipt will read `9.60` like a grown-up register.

Until then, The Daily Grind prints ugly totals *on purpose*, as a reminder. *(Keep this slide in mind for the exam: float imprecision is a classic question.)*

How big? How small?

Integers in Python are **unbounded**:

```
>>> print(1000 ** 103)        # a 1 with 309 zeros, no problem
```

Floats are not; their binary format has limits:

```
>>> print(1000.0 ** 102)      # 1e+306         (scientific notation)
>>> print(1000.0 ** 103)      # OverflowError: (34, 'Result too large')
>>> print(0.1 ** 323)         # 1e-323         (almost nothing)
>>> print(0.1 ** 324)         # 0.0            (quietly *underflows* to zero!)
```

Examinable trivia with a real point: **int = exact and unlimited; float = fast and approximate.** Choose knowingly.

Booleans

The café is either open or it isn't:

```
>>> open_sign = True
>>> print(type(open_sign))
<class 'bool'>
>>> print(type(true))
NameError: name 'true' is not defined
```

- Exactly two values: `True` and `False`, **capitalized** (case sensitivity again).
- They secretly behave as integers: `True = 1`, `False = 0`: `False + False` → `0`, `True * True` → `1`, `False - True` → `-1`, and `True / False` → … `ZeroDivisionError`. Be careful with boolean arithmetic.

§8 · TRY IT The bool() quiz (2 min)

Everything can be converted to a boolean. Predict each one:

```
print(bool(0))            # ?
print(bool(1000000))      # ?
print(bool(0.001))        # ?
print(bool(-0.001))       # ?
print(bool("Hello!"))     # ?
print(bool("False"))      # ?   <- the sneaky one
print(bool(""))           # ?
```

Rule: zero and empty are `False`; **everything else is `True`:** including the *text* `"False"`, which is a non-empty string. This exact trick shows up on exams.

§9 · CONCEPT Variables

```
>>> x = 5
>>> print(x)
5
```



- A variable **attaches a name to a value:** like the M+ / MR buttons on a calculator.
- An assignment has two sides: **RHS is computed first**, then the name is attached: `x = 5 / 2` → the *result* `2.5` is stored, and `x` points to it.
- Python detects the type for you: that `x` is a `float` now.

```
>>> a_laugh = "ha"
>>> intensity = 3
>>> print(a_laugh * intensity)
hahaha
```

§9 · CONCEPT Naming: rules, then taste

Rules: letters, digits and _; may not start with a digit; may not be a **keyword**; all thirty-five of them, for the record:

```
False     None      True      and       as        assert    async     await
break     class     continue  def       del       elif      else      except
finally   for       from      global    if        import    in        is
lambda    nonlocal  not       or        pass      raise     return    try
while     with      yield
```

(You'll meet most of these as the term unfolds; today they're just names you can't take.)

Taste (this is the part that gets read by humans):

```
x = 3.14                                  # too little (ok in passing)
pi = 3.14                                 # good
variable = 3.14                           # useless
the_value_of_pi_which_is_3_dot_14 = 3.14  # please, no
```

Our café: `espresso_price`, not ep. `open_sign`, not `flag2`. **A name is the cheapest documentation you'll ever write.**

§9 · TRACE Rebinding: follow the arrows

Run this by hand. What prints?

```
x = 1
y = x + 3.14
x = 7
print(y)
```

`4.140000000000001` (the float wall, again), not `10.14`. Line 2 computed `y` from the value `x` had *at that moment*. Rebinding `x` later doesn't reach back.

And memory? When `x = 7` replaces `x = 1`, nothing points at the old `1` anymore, so Python deletes it: **automatic garbage collection**. You can also evict manually: `del(x)`, after which `print(x)` raises `NameError`.

Updating in place

“Add this to the total” is so common it has shorthand, augmented assignment:

```
revenue = 0.0
revenue += 7.40      # same as: revenue = revenue + 7.40
```

The whole family: `+=` `-=` `*=` `/=` `**=` `%=` `//=`.

Tonight’s receipt ends with one: `revenue += total`. You will read `+=` in every Python codebase you ever open; our own café then writes the step out in full (`revenue = revenue + paid`), so a trace can show both sides of it.

Types & conversions

Four types so far, and `type()` reads them:

`"5"` is a `str` · `5` is an `int` · `3.14` is a `float` · `False` is a `bool`.

Convert between them with `str()`, `int()`, `float()`, `bool()`:

```
>>> x = "5"
>>> x = int(x)        # "5" (text) -> 5 (number)
```

Careful: `int(x)` doesn't change what you pass in. It *builds a new object*. Above, `x = int(x)` rebinds `x` to it; skip the assignment and the new value is gone. (`print(int(3.14))` shows `3`, leaving the `3.14` itself untouched.)

The one you'll hit tonight

```
>>> x = 100
>>> print("I am " + x + " years old.")
TypeError: can only concatenate str (not "int") to str
```

`+` between a string and a number is **undefined:** Python refuses to guess whether you meant text-joining or addition. The fix is to convert first:

```
>>> print("I am " + str(x) + " years old.")
I am 100 years old.
```

This is the key to our receipt: every number that goes into printed text goes through `str()` first. (Week 2 brings a much nicer way; one more reason to come back.)

§10 · CAFÉ The receipt: the ask & the plan

> *"Print me a receipt: the shop's name, each item with quantity and line total, the total, and whether we're open."*

Week-1 decomposition (no loops, no functions yet, just top-to-bottom):

1. name the café; set the prices *(variables)*;
2. set the quantities; compute line totals and the total *(arithmetic)*;
3. print a divider, the lines, the total *(strings, `*`, `+`, `str()`)*.

Design choice worth saying out loud: prices as **floats:** we know the risk now, and we accept it *knowingly* until week 2 fixes the display.

§10 · CAFÉ The receipt: build

```
cafe_name = "The Daily Grind"
open_sign = True

espresso_price = 2.50
muffin_price = 2.40

espresso_qty = 2
muffin_qty = 1
total = espresso_price * espresso_qty + muffin_price * muffin_qty

print("=" * 34)
print(cafe_name + ": receipt")
print("=" * 34)
print("espresso  x " + str(espresso_qty) + "    " + str(espresso_price * espresso_qty))
print("muffin    x " + str(muffin_qty) + "    " + str(muffin_price * muffin_qty))
print("TOTAL: " + str(total))
print("Open: " + str(open_sign))
```

Full file: `cafe_w1.py`. Every line is something from today.

§10 · CAFÉ Test & reflect

```
==================================
The Daily Grind: receipt
==================================
espresso  x 2   5.0
muffin    x 1   2.4
TOTAL: 7.4
Open: True
```

It **works**. And it's **limited:** feel exactly where:

- the line totals print as `5.0` and `2.4`, not `5.00` / `2.40` *(formatting: week 2)*;
- the order is **hard-coded**; no one can type what they want *(input: week 2)*;
- three lattes would print `9.600000000000001` *(we know why!)*;
- every new product means copy-pasting lines *(structures: week 4)*.

Each limitation is a **coming attraction**.

Where the café is now

Runnable snapshot, **`cafe_w1.py`:** name, open sign, three prices, one hard-coded order, a printed receipt, and one *deliberate* ugly total as a souvenir of the float wall.

Next week the café learns to **listen and remember**: keyboard input, proper formatted output (goodbye `9.600000000000001`), its first **functions**, and saving the day's revenue to a **file** that survives closing time.

Exam pointers

The exam can ask you to:

- state the output of `print` / arithmetic / conversion snippets (precedence traps, `bool("False")`, string `*` and `+`);
- explain float imprecision (`0.1 + 0.1 + 0.1`) in one sentence;
- spot the error: `Print(...)`, mismatched quotes, `"text" + number`, `true`;
- theory: machine vs assembly vs high-level; what a string is underneath.

Exercises & self-study

(optional; full map on Brightspace):

- **Self-check 1** → Brightspace tonight: about fifteen anonymous minutes, closes Thursday 23:59. We open next week on whatever the class lost.
- **Practical sheet** → Brightspace (the week-2 practical works through it).
- **Project groups:** the group form is open on Brightspace (five from one practical session; details in the overview deck). Deadline: the **end of week 5**. Early birds pick their teammates.
- **py4e** *(free, recommended)*: *Installing Python* + Ch 1–2: read, **watch the chapter videos**, do the **auto-graded exercises**.
- **ATBS** *(free alternative)*: Intro + Ch 1. · **LPTHW** *(if you own it)*: Ex 0–4.
- *Heads-up:* a few of their exercises use `input()`; that's **next week's** material; park those until Monday, or peek ahead.

This week's homework: install Python 3 + VS Code; the first practical is Monday of week 2, and it starts from your keyboard, not mine.

This week's checklist: tick before the practical

- ☐ I can explain what a program is, and how machine code, assembly and high-level languages relate.
- ☐ I can run Python three ways (interpreter, `.py` file, editor) on my own machine.
- ☐ I can `print()` text, use escape characters, and comment with `#`.
- ☐ I can compute with `+ - * / % ** //` and predict the precedence.
- ☐ I can explain float imprecision with `0.1 + 0.1 + 0.1`, in one sentence.
- ☐ I can predict what `bool()` says about a value, and why `True == 1`.
- ☐ I can use variables (legal names, good names), `type()`, and conversions, and I know why `"text" + number` crashes.
- ☐ **Setup done:** Python 3 + VS Code installed and working.

Anything unticked → that section of this deck, then the practical sheet.

Next week: the wall falls: f-strings and formatting, `input()`, files; the register starts listening and remembering.